Enterprise Student Record and Academic Management Platform

Unit 1: Product Engineering & Design Thinking
Careertiq Training & Engineering Division

1. Introduction to Software Product Development

In modern organizations, software products are built using structured engineering frameworks rather than ad-hoc programming. Coding is merely one execution phase within a larger engineering lifecycle that spans research, architecture, user experience design, quality assurance, automated deployment, and continuous monitoring.

1.1 Comparative Analysis: Programming vs. Software Engineering

Dimension	Programming / Coding	Software Product Engineering
Primary Goal	Translating logic/ algorithms into executable code syntax.	Building scalable, maintainable, and valuable systems for end users.
Scope	Isolated functions, micro-tasks, or single scripts.	Full lifecycle (SDLC), architecture, CI/CD, database schemas, security.
Success Metric	Code compiles and passes basic test inputs.	User adoption, high availability, performance metrics, business outcome.
Mindset	Technology-Centric ("How do I write this function?").	Product-Centric ("What core problem are we solving for our users?").

1.2 Root Causes of Software Project Failures

Software projects rarely fail due to bad syntax. Major failure vectors include:

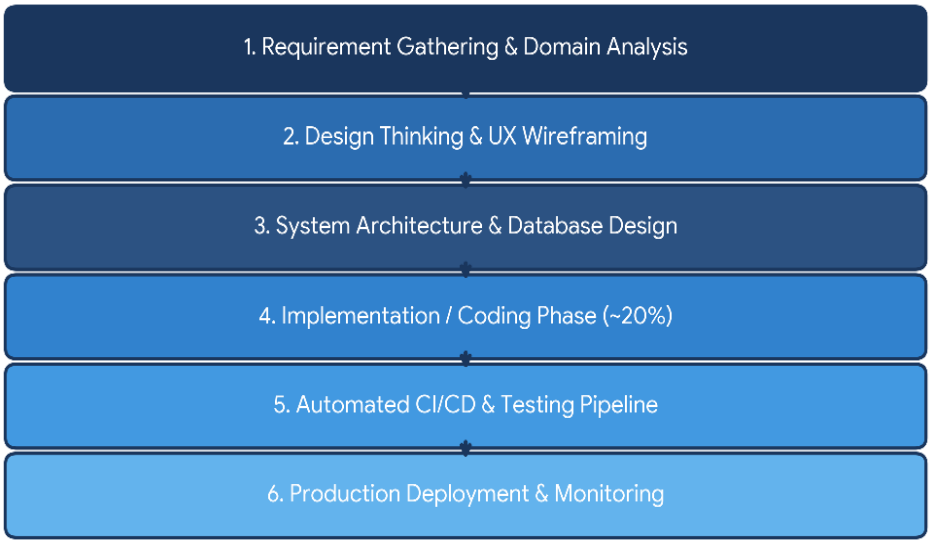
Misaligned Requirements: Building features that users do not need or want.

Technology-Centric Bias: Choosing technologies based on industry hype rather than suitability for business domain problems.

Communication Silos: Separation between business analysts, software developers, and DevOps infrastructure teams.

Lack of Iterative Feedback: Delaying user testing until late development stages, making pivot costs prohibitively high.

System Development Lifecycle (SDLC) Workflow Diagram



2. Business Understanding: Higher Education & Academic Operations

Before writing code or provisioning infrastructure, engineers must master the business operations of the target environment.

2.1 Core Operations in Academic Institutions

Student Onboarding & Profile Management: Enrolling students, capturing permanent registration numbers (PRN), establishing degree paths, and managing demographic data.

Course Enrollment & Curriculum Mapping: Allocating course electives, managing semester course offerings, and assigning faculty subject tracks.

Attendance Tracking Workflow: Recording daily lecture logs, calculating attendance percentages, generating threshold warnings, and logging leave requests.

Examination & Grading Operations: Conducting continuous assessments, processing end-semester marks, calculating GPA/CGPA rubrics, and executing revaluation workflows.

Administrative & Compliance Reporting: Generating consolidated grade sheets, transcript verification, audit logs, and institutional accreditation reports.

2.2 Manual Process Challenges vs. Automated Software Solutions

Manual Operational Bottleneck	Automated Platform Solution	Business Outcome Metric
Physical paper ledgers for daily lecture attendance	Mobile and web-based continuous attendance logging	Attendance tracking time reduced from 15 min/class to <1 minute
Manual calculation of semester GPA/CGPA grades	Engine-driven grading pipeline with configurable rubrics	Result generation speed improved by 90%
Physical mark sheet distribution and transcript tracking	Centralized student self-service portal for real-time grade publishing	Operational transcript issuance delays reduced by 70%
Siloed records leading to lost student files & reporting errors	Relational database with strict integrity and audit constraints	Academic record accuracy brought to 99.9%

3. Introduction to Design Thinking

Design Thinking is an iterative, human-centered problem-solving framework that prioritizes deep empathy for users before evaluating technological constraints.

4. Applying the Five Stages of Design Thinking



Stage 1: Empathize

Engineers and designers observe users in their daily academic workflows. Key user categories analyzed include: Students, Faculty Members, Academic Administrators, Examination Cell Staff, and IT Personnel.

Stage 2: Define

Data collected during empathy building is organized into a definitive core problem statement:

CORE PROBLEM STATEMENT:
"Students and administrative personnel experience severe operational delays in handling course enrollments, attendance tracking, mark publishing, and official transcript validation due to fragmented manual record handling. Concurrently, faculty and examination cell staff lack real-time centralized visibility, leading to high error rates, compliance risks, and administrative friction."

Stage 3: Ideate

Unconstrained feature brainstorming yields high-value capabilities: Automated low-attendance notifications via SMS/email, digital student identity cards, faculty grading portals, real-time CGPA progress trackers, and dynamic transcript generators.

Stage 4: Prototype

Low-fidelity wireframing of core UI modules: Student Portal Dashboard, Faculty Attendance Entry Page, Exam Cell Grading Panel, and Administrative Analytics Hub.

Stage 5: Test

Paper prototypes and interactive mockups are demonstrated to students, professors, and academic deans to evaluate workflow clarity, layout accessibility, and task completion speed prior to technical implementation.

5. Enterprise Expansion: IBM Design Thinking

IBM Design Thinking scales human-centered design for large-scale enterprise delivery through three core pillars:

Focus on User Outcomes: Shifting technical goals to measurable human metrics.

• Feature Goal: "Build a student database query endpoint."

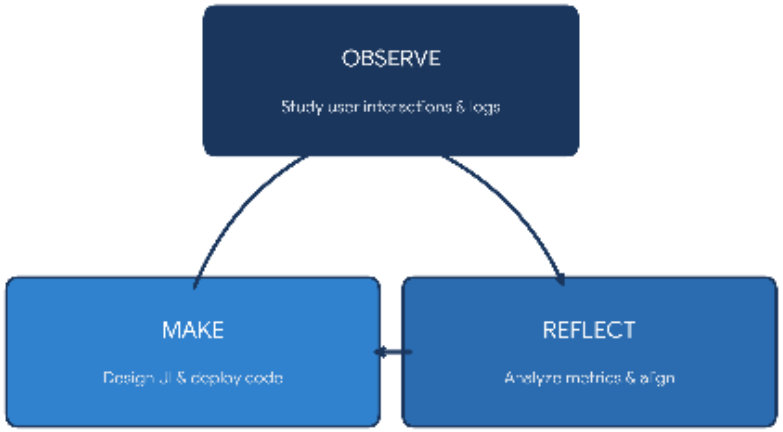
• Outcome Focus: "Enable academic advisors to lookup a student's full 4-year grade transcript in under 2 seconds."

Restless Reinvention: Treating software continuous deployment (CD) as an ongoing refinement cycle—constantly analyzing production usage to optimize UX patterns and database response latency.

Diverse Empowered Teams: Cross-functional collaboration across Product Managers, Front-End Engineers, Database Architects, Exam Cell Domain Experts, and DevOps Teams.

6. The IBM Loop Engine (Observe -> Reflect -> Make)

The platform is engineered using the continuous cycle of Observe -> Reflect -> Make.



7. Stakeholder User Personas

User Persona	Primary Objectives	Key Frustrations & Pain Points	Target Platform Feature
Priya (Student)	Track real-time attendance, view semester results instantly, apply for electives.	Lack of attendance transparency; delayed result publications.	Responsive student dashboard, grade visibility portal, attendance alerts.
Prof. Sharma (Faculty Member)	Record daily lecture attendance easily; enter continuous evaluation marks.	Tedious paper register management; double entry of marks during finals.	Quick-click attendance logger, bulk grade entry tool, student deficit tracker.
Dr. Kapoor (Exam Cell Admin)	Process institutional results securely; generate transcripts and compliance audits.	High risk of manual mark entry errors; long processing cycles for grade cards.	Automated GPA engine, RBAC secure grading workflows, 1-click transcript export.

8. Stakeholder Matrix

Stakeholder Group	Key Roles	Responsibilities & Project Expectations
Business Stakeholders	Academic Deans, Department Heads, Students, Registrar Office	Define institutional goals, validate workflow adherence, ensure compliance with education standards.
Engineering Stakeholders	Software Engineers, UI/UX Designers, QA Specialists	Architect system backend, construct intuitive user interfaces, ensure test coverage and security.
Operations Stakeholders	System Administrators,	Maintain database uptime,

	Database Administrators, DevOps Team	enforce role-based access control (RBAC), automate deployment pipelines.

9. Requirements Engineering Specification

9.1 Functional Requirements (FRS)

FR-01 Authentication & RBAC: Secure Role-Based Access Control enforcing permissions for Student, Faculty, Academic Admin, and Exam Cell roles.

FR-02 Student Profile & Admission Management: Complete profile maintenance, personal details, PRN assignment, and course mapping.

FR-03 Course & Enrollment Engine: System capability for students to register for active semester electives and core courses.

FR-04 Attendance Management Module: Digital attendance entry interface for faculty with real-time percentage calculation and dynamic low-attendance alerts.

FR-05 Examination & Result Engine: Automated processing of internal and external marks to generate SGPA/CGPA with instant PDF report generation.

9.2 Non-Functional Requirements (NFRS)

NFR-01 Security: Industry-standard password encryption (Bcrypt/Argon2), JWT token session handling, HTTPS encryption, and strict RBAC isolation.

NFR-02 Performance: Sub-500ms API response time for record lookups and mark submissions under peak usage.

NFR-03 Scalability & Availability: Cloud-native modular architecture guaranteeing 99.9% availability during peak exam result query traffic.

NFR-04 Usability & Accessibility: Fully responsive UI design complying with WCAG 2.1 AA standards for seamless access on desktop and mobile devices.

10. Product Vision & Minimum Viable Product (MVP) Scope

PRODUCT VISION STATEMENT:
"To deliver a unified, enterprise-grade Student Record and Academic Management Platform that eliminates manual administrative overhead, accelerates result publishing, and empowers academic communities with seamless, secure access to real-time educational metrics."

MVP Release Scope (Sprint 1–4)

To maintain agility, non-essential capabilities (such as alumni portal integration, hostel management, or AI performance predictions) are deferred. The MVP includes:

Secure User Login & RBAC Role Provisioning

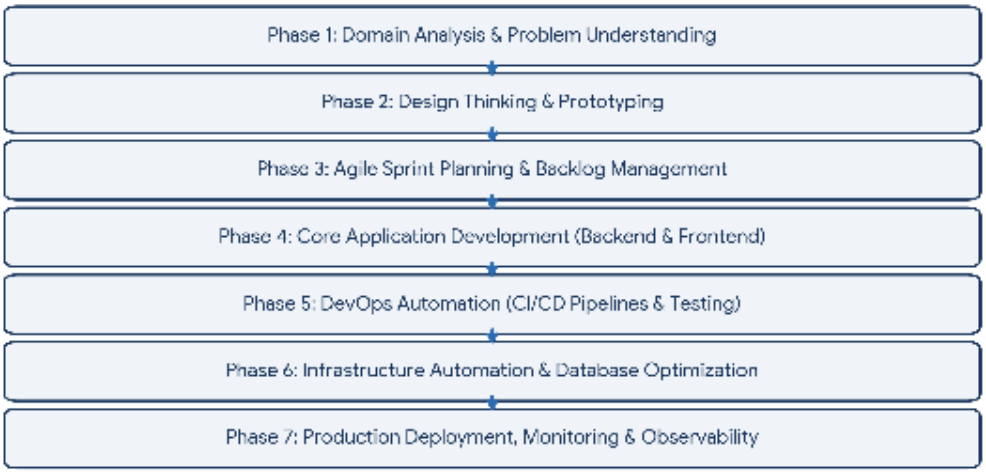
Student Profile & Course Registration Engine

Daily Digital Attendance Entry & Percentage Tracker

Assessment Marks Processing & SGPA Calculation

Standard Academic Transcript & Report Generation

11. Enterprise Engineering Roadmap



12. System Architecture & Repository Structure

Repository Hierarchy

```
student-record-academic-platform/
    .github/          # CI/CD Workflows (GitHub Actions)
    app/              # Application Source Code
        modules/      # Business Logic (Students, Attendance, Marks, Courses)
        database/     # Schemas, Migrations, & ORM Models
        tests/        # Automated Unit & Integration Tests
    docs/             # Architecture Diagrams, FRS/NFRS Specs, & Design Reports
    docker/           # Dockerfiles & Multi-Container Compose Setup
    kubernetes/       # K8s Manifests (Deployments, Services, Ingress)
    terraform/        # Cloud Infrastructure Provisioning Scripts
```

rainer Execution Deliverables Checklist

- [x] Business Problem Statement & Business Justification Report
- [x] Product Vision Statement & MVP Feature Matrix
- [x] Stakeholder Responsibility Matrix & User Persona Profiles
- [x] Functional (FRS) and Non-Functional (NFRS) Requirements Specs
- [x] Design Thinking Report (Empathize, Define, Ideate, Prototype, Test)
- [x] IBM Design Thinking & IBM Loop Execution Specs
- [x] Multi-Phase Product Engineering Roadmap
- [x] System Architecture Diagram Concept Specs
- [x] Initialized Git Repository & Standardized Folder Architecture

13. Visual Flowcharts and System Diagrams

The following diagrams provide a visual representation of the product discovery, design thinking, academic workflow, system architecture and IBM continuous improvement cycle. Flowcharts are useful for communicating process steps and data flow, while architecture diagrams show system components and their relationships.
cite turn0search0 turn0search2

13.1 Design Thinking Process Flowchart

This flowchart summarizes the five stages applied to the student academic management problem.

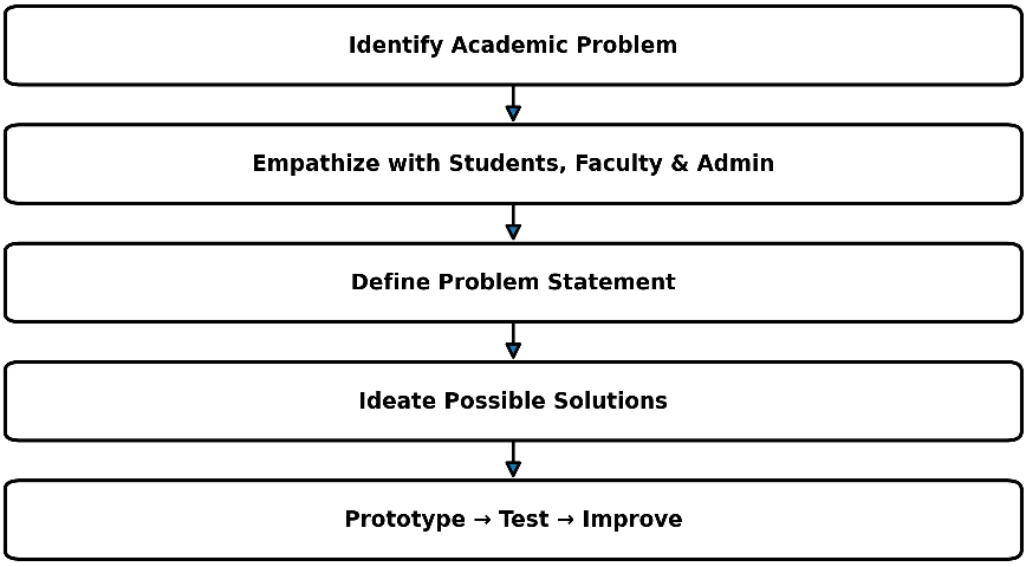


Figure 13.1: Design Thinking Process Flowchart

13.2 Initial System Architecture Diagram

The architecture connects the main user groups to the interface, application/API layer, academic modules and centralized database.

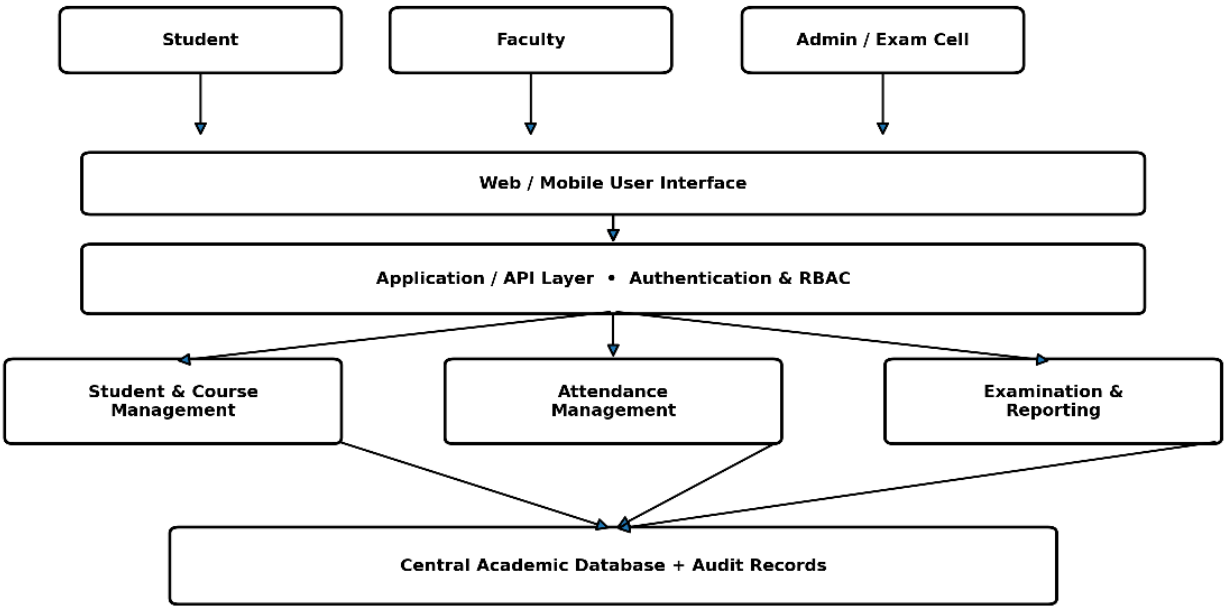


Figure 13.2: Initial System Architecture

13.3 Student Academic Management Workflow

The workflow shows how academic information moves from student admission and course registration through attendance and examination processing to academic reports.

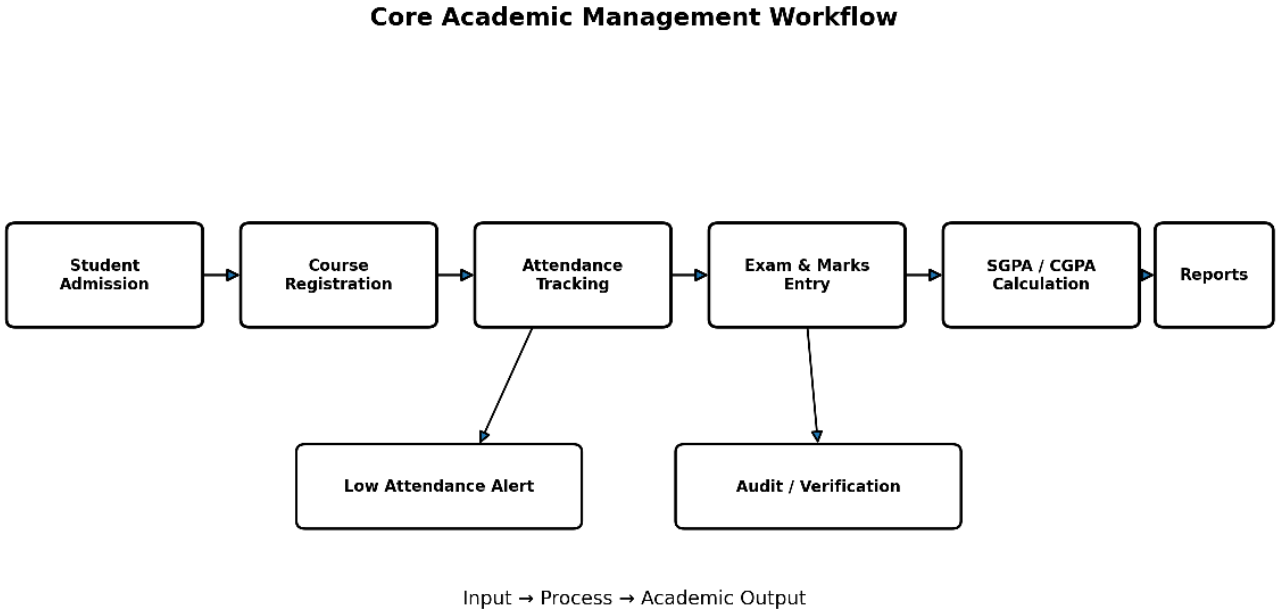


Figure 13.3: Core Academic Management Workflow

13.4 IBM Design Thinking Loop

The loop represents continuous observation, reflection, making and feedback for iterative product improvement.

